

Q1: You are a Machine Learning Engineer at a company that manages wind farms. Your task is to build a binary classification model (Neural Network) to predict whether a wind turbine gear is about to fail (1) or is healthy (0) based on sensor data.

You have a dataset of 10,000 readings. Each reading has two input features:

- Vibration Frequency: Measures in Hertz (Hz), ranging from 0.001 to 0.05 Hz.
- Rotational Speed: Measures in RPM, ranging from 0 to 3,500 RPM.

- i. You feed the raw data directly into a simple Neural Network. During training, you notice the model struggles to converge (the loss goes down very slowly). The gradients related to Rotational Speed oscillate wildly, while the gradients for Vibration Frequency barely move.

Why is the model struggling to learn from both features effectively? Which technique should you apply to mitigate this, and how would it transform the data?

- ii. After fixing the data issue in part (i), you train two different models.
 - Model A is a very shallow network (1 hidden layer, 2 neurons). It achieves 60% accuracy on the training set and 59% accuracy on the test set. (Note: A random guess is 50%).
 - Model B is a massive network (10 hidden layers, 1024 neurons each). It achieves 99.8% accuracy on the training set but only 65% accuracy on the test set.

Identify and explain the issues with these two models.

- iii. You decide to stick with the architecture of Model B (the large network) because you believe the problem is complex, but you need to fix the generalization gap. You introduce a technique called Dropout with a rate of $p=0.5$.

Describe what happens to the neurons in the hidden layers during a single training pass (forward and backward propagation) when Dropout is active. How does this force the turbine model to become more robust?

- iv. You are now training your optimized model. You plot the Training Loss and Validation Loss over 1000 epochs.
- Epoch 0 to 200: Both Training and Validation loss decrease steadily.
 - Epoch 200: Validation loss hits its lowest point (0.25).
 - Epoch 201 to 1000: Training loss continues to drop to nearly 0.01 and level off, but Validation loss starts creeping up, eventually reaching 0.8.

If you strictly minimize Training Loss, you will stop at Epoch 1000. If you use Early Stopping, at which Epoch would you stop training? What is the danger of continuing past that point? In actual coding, what is the parameter to tweak to prevent the training from stopping prematurely due to noise in validation metrics?

- v. You are preparing to train your finalized Neural Network on the 10,000-sample dataset. You need to configure the training loop hyperparameters. You decide to use Mini-Batch Gradient Descent with a Batch Size of 100. You plan to run the training for 50 Epochs. Based on the numbers above, calculate the following values. Show your working to distinguish between the terms.
- Iterations per Epoch: How many "steps" (updates) does the model make to get through the entire dataset once?
 - Total Training Iterations: How many times will the model weights be updated during the entire 50-epoch training process?
 - If you changed the Batch Size to 10,000 (Batch Gradient Descent), how many total weight updates would occur over 50 epochs?
 - If you changed the Batch Size to 1 (Stochastic Gradient Descent), how many total weight updates would occur over 50 epochs?

vi. Let's zoom in on a single specific weight in your neural network, let's call it weight w_1 .

- Current Value: $w_1 = 2.50$
- Learning Rate (α): 0.1
- Batch Size: 4 samples (simplified for this calculation)

During one iteration, the network processes this batch of 4 samples. For each sample, it calculates the gradient (the recommended change) for weight w_1 to reduce the error for that specific sample.

- Sample 1 Gradient g_1 : +0.8 (Suggests increasing weight)
- Sample 2 Gradient g_2 : -0.2 (Suggests decreasing weight slightly)
- Sample 3 Gradient g_3 : +0.6 (Suggests increasing weight)
- Sample 4 Gradient g_4 : +1.2 (Suggests increasing weight significantly)

Calculate the **Average Gradient** for this batch.

Calculate the **New Value** of weight w_1 after this iteration is complete.

Explain **when** the update happens: Does w_1 change four times (after each sample), or only once?

Q2: You are the Lead Data Scientist at a hospital. You have developed an AI model called "MediScan" to detect a rare but treatable disease called Syndrome X. You test the model on a fresh batch of 1,000 patients.

- The Reality: In this group, only 50 patients have the disease. The other 950 are healthy.
 - The Model's Performance:
 - o The model correctly identified 40 of the sick patients.
 - o The model missed 10 sick patients, saying they were healthy.
 - o The model correctly said 860 healthy people were healthy.
 - o The model incorrectly flagged 90 healthy people as having the disease.
- i. Map the story above into the four quadrants of the Confusion Matrix, i.e. compute the TP, TN, FP and FN. What do these four values represent?
- ii. Calculate the Accuracy, Recall and Precision metrics.
- iii. You present your results to the Hospital Board. The Hospital Administrator looks at the Accuracy and says: "Wow! 90% accuracy! This model is excellent, let's deploy it immediately." Explain why the Administrator is wrong. Why is the statement "90% accuracy" misleading in this specific case?
- iv. Identify and explain which metric (Accuracy, Recall or Precision) should be used to evaluate model performance for the two scenarios below.
- a. Scenario A (The Deadly Contagion): Syndrome X is highly contagious and fatal if not treated early. The treatment is just a simple pill with no side effects.
 - b. Scenario B (The Risky Surgery): Syndrome X grows very slowly and is not contagious. However, the treatment involves risky, expensive open-heart surgery.
- v. You want a single metric (F1) that balances both concerns (Precision and Recall) so you don't have to look at two numbers. If you improved your model so that Precision became 1.0 (perfect) but Recall dropped to 0.1, would the F1 score go up or down? (Qualitative answer).